RESTful Web Services and WebSockets

Juan Luis Gorricho

RESTful web services

- Representational State Transfer (REST) has gained widespread acceptance across the Web as a simpler alternative to SOAP and WSDL based Web services deploying services on the Internet, due to its lightweight nature and the ability to transmit data directly over HTTP.
- The RESTful architecture is focused on managing resources instead of the usual RPC implementation.
- The HTTP commands commonly used are: GET, POST, PUT and DELETE to materialize the usual CRUD actions on resources.

RESTful web services

- Representational State Transfer means that the state of the user is transferred along the same HTTP request as a representational form.
- Consequently, statelessness means that every HTTP request happens in complete isolation. When the client makes an HTTP request, it includes all information necessary for the server to fulfill that request.

RESTful web services

- By stateless it means that the **web server** does not store any state about the client.
- That is where the *ST* in *REST* comes from, *State Transfer*. You transfer the state around, instead of having the server store it. **This is the only way to scale to millions of users.**
- The load of session management is amortized across all the clients, the clients store their session state and the servers can service many orders of magnitude or more clients in a stateless fashion.

JSON

- JSON or JavaScript Object Notation, is an open standard format that uses human-readable text to transmit data objects consisting of attribute–value pairs.
- It is used primarily to transmit data between a server and a web client application, as an alternative to XML.
- Although originally derived from the JavaScript scripting language, JSON is a language-independent data format.
- Code for parsing and generating JSON data is readily available in many programming languages.

GSON

- Gson (also known as Google Gson) is an open source Java library to serialize and deserialize Java objects to (and from) JSON data structures. Code samples:

```
Gson gson   = new Gson();
String json   = gson.toJson(an_object);
Myclass obj = gson.fromJson(json, Myclass.class);
Myclass[] array = gson.fromJson(json, Myclass [].class);
```

RESTful web services and NetBeans

- In order to understand the programming of RESTful web services with NetBeans we will create a new web app.
- Define, as usual, an "entity" package.
- From that package, right-click on the mouse and select the entry: "Restful Web Services from Database". Create a new Data Source using the "ecommerce" ddbb connection.
- Try to understand all the code that has been generated by the NetBeans assistant.

URLs

```
Interface Comms {
    String PATH = "localhost:8080/uppercaseServer";
    String SERVER_REST = "http://"+PATH+"/webresources";
    String SERVER_WEBSOCKET = "ws://"+PATH+"/ws";
}
```

apiREST

```
URL url = new URL(Comms.SERVER_REST + "/entity.myentity/");
HttpURLConnection ucon = (HttpURLConnection) url.openConnection();

ucon.setRequestMethod("POST");
ucon.setDoInput(true);
ucon.setDoOutput(true);
ucon.setRequestProperty("Content-Type", "application/json; charset=utf-8");
ucon.setRequestProperty("Accept", "application/json; charset=utf-8");

PrintWriter out = new PrintWriter(ucon.getOutputStream(), true);
String json = new Gson().toJson(sentence);
out.println(json);
out.flush();
ucon.connect();

BufferedReader in = new BufferedReader(new InputStreamReader(ucon.getInputStream()));
String line;
while ((line = in.readLine()) != null) {
    System.out.println(line);
}
```

Programming with WebSockets

- Snippet of code for servers:

```
@ServerEndpoint("/ws")
public class WebSocketServer {

    private static Set<Session> sessions = new HashSet<Session>();

    @OnOpen
    public void onOpen(Session session) {...}

    @OnClose
    public void onClose(Session session) {...}

    @OnMessage
    public void onMessage(String message, Session session) {...}

    //any method
    public void myMethod(String json , Session session) {
            session.getBasicRemote().sendText(json);
    }
}
```

Programming with WebSockets

- Snippet of code for clients:

```
@ClientEndpoint
public class WebSocketClient {

   static Session session;

   public static void newInstance() {
       WebSocketContainer container = ContainerProvider.getWebSocketContainer();
       session = container.connectToServer(WebSocketClient.class, URI.create(Cons.SERVER_WEBSOCKET)); }

   @OnMessage
   public void onMessage(String message) {...}

   //any method
   public void myMethod(String json) {
       session.getBasicRemote().sendText(json);
   }
}
```

Libraries to be used

- At the client side:
  - gson-2.2.4.jar
  - json-org.jar
  - tyrus-standalone-client-1.3.3.jar (WebSocketClient)
- At the server side:
  - gson-2.2.4.jar
  - json-org.jar

Warm up exercise

- Create a data base with a single table to store sentences.
- Program a new webApp:
  - using RESTful web services from that data base. Modify the façade to store the sentences always in upper case.
  - Add the programming of a WebSocketServer to be able to send the stored sentences to all connected WebSocket clients when needed.
- Complete the code of the client accordingly.